

Problem Solving with Decisions

CHAPTER 5

Overview

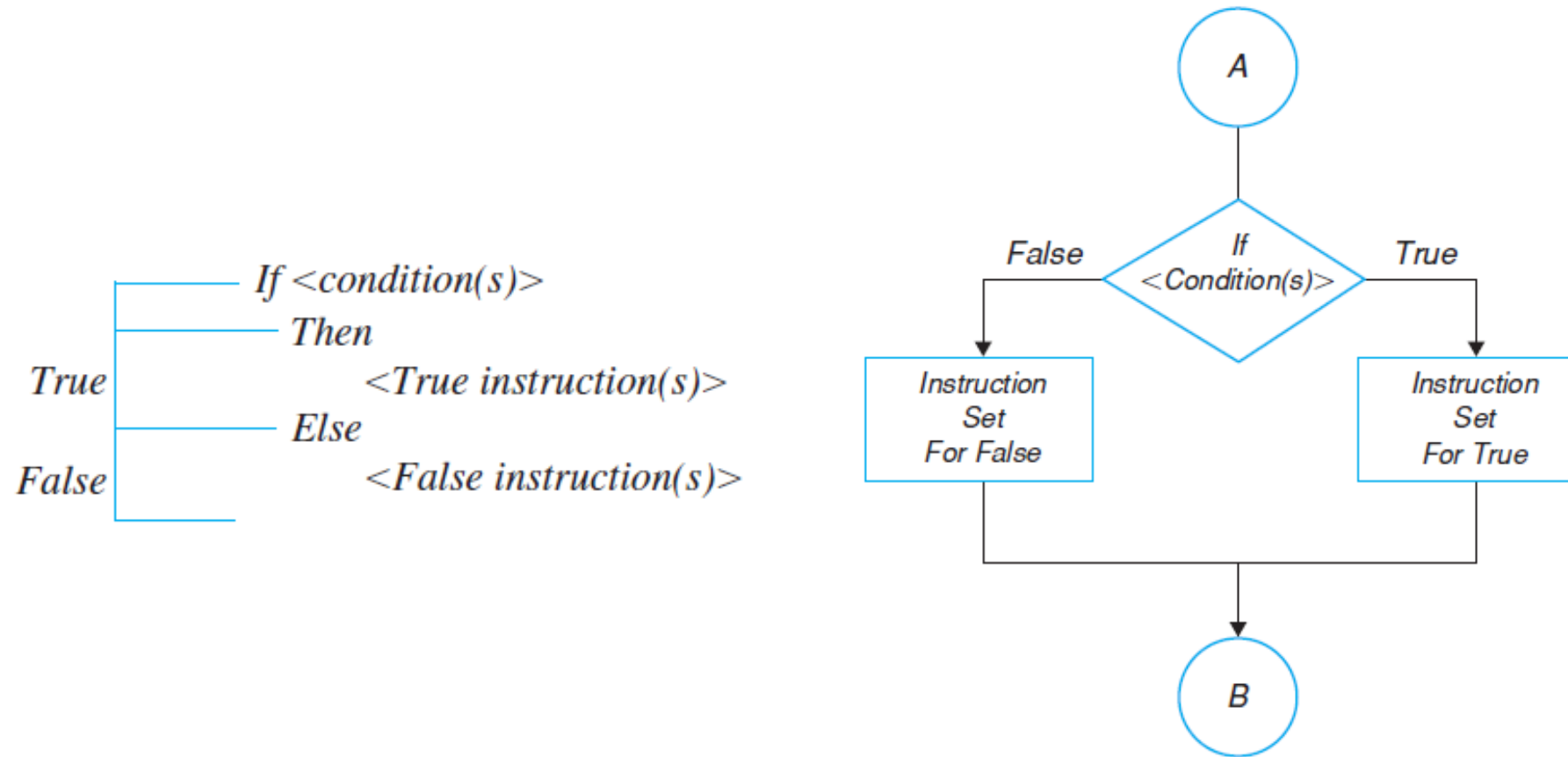
- **Decision Logic Structure**
- **Single If/Then/Else Instruction**
 - One Condition
 - Single Selection IF/Then
 - Double Selection If/Then/Else
- **Multiple Condition**
- **Multiple If/Then/Else Instructions**
 - Straight-Through logic
 - Positive Logic
 - Negative logic
 - Logic conversion
- **Decision Table**



Decision Logic Structure

- The decision structure is one of the most powerful structures because it is the only way that the computer can choose between two or more sets of actions.
- The decision logic structure uses the **If/Then/Else** instruction.
- It tells the computer that If a condition is true, Then execute a set of instructions, or Else execute another set of instructions.
- The Else part is optional.

Common Form of decision structure



Decision Logic Structure

- **A condition can be one of four things:**

- 1- A logical expression, that is, an expression that uses logical operators (AND, OR, and NOT)
- 2- An expression using relational operators (<, <=, >, >=, =, and < >).
- 3- A variable of the logical data type (True, False).
- 4- A combination of logical, relational, and mathematical operators.

Decision Logic Structure

- Some examples of conditional expressions are as follows:

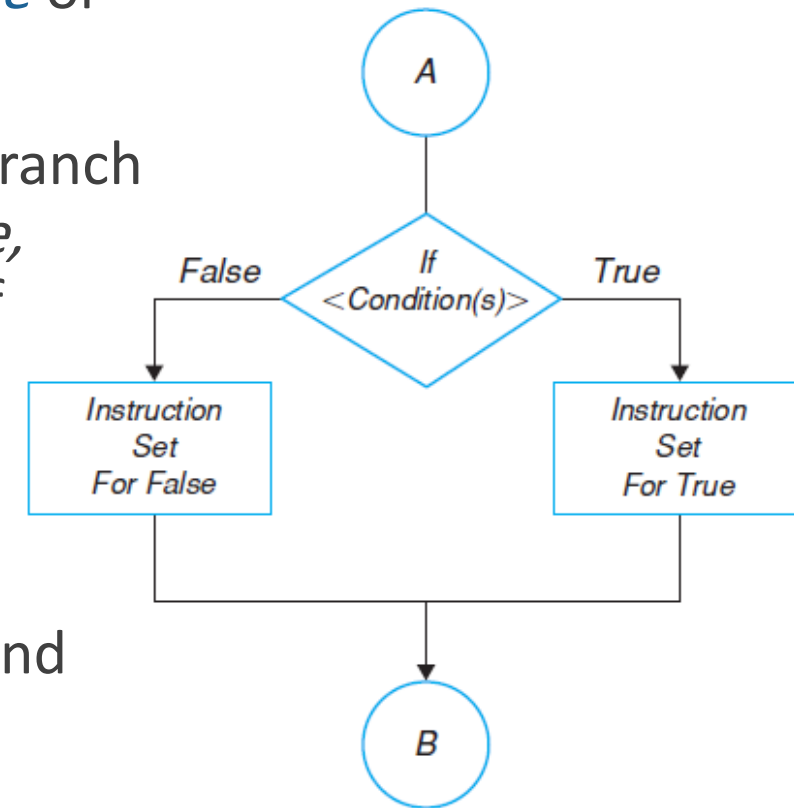
1. $A < B$ (A and B are the same data type—either numeric, character, or string)
2. $X + 5 \geq Z$ (X and Z are numeric data)
3. $E < 5 \text{ OR } F > 10$ (E and F are numeric data)
4. DataOk (DataOk is a logical datum)

Decision Logic Structure

- Logical operators are used to link more than one condition together.
For example: (Check < 50 **AND** DriverLicense) note: the data type of DriverLicense is logical.
- The programmer can set up the conditions for a decision through the use of operands and various operators.
- These conditions can be used alone or linked with other conditions for use in the instruction.
- The programmer derives the information needed to set up the condition(s) for a decision from the problem itself during problem analysis

Decision Logic Structure

- In the figure, the **resultant** of the condition(s) can be *True* or *False* depending on the data.
- From the **decision block (the diamond)**, there will one branch for the instructions to be executed if the resultant is *True*, and another branch for the instructions to be executed if the resultant is *False*.
- The True branch and the False can come from any of the lower three points of the diamond.
- The **convention** is to draw the True branch on the right and the False branch on the left or bottom.

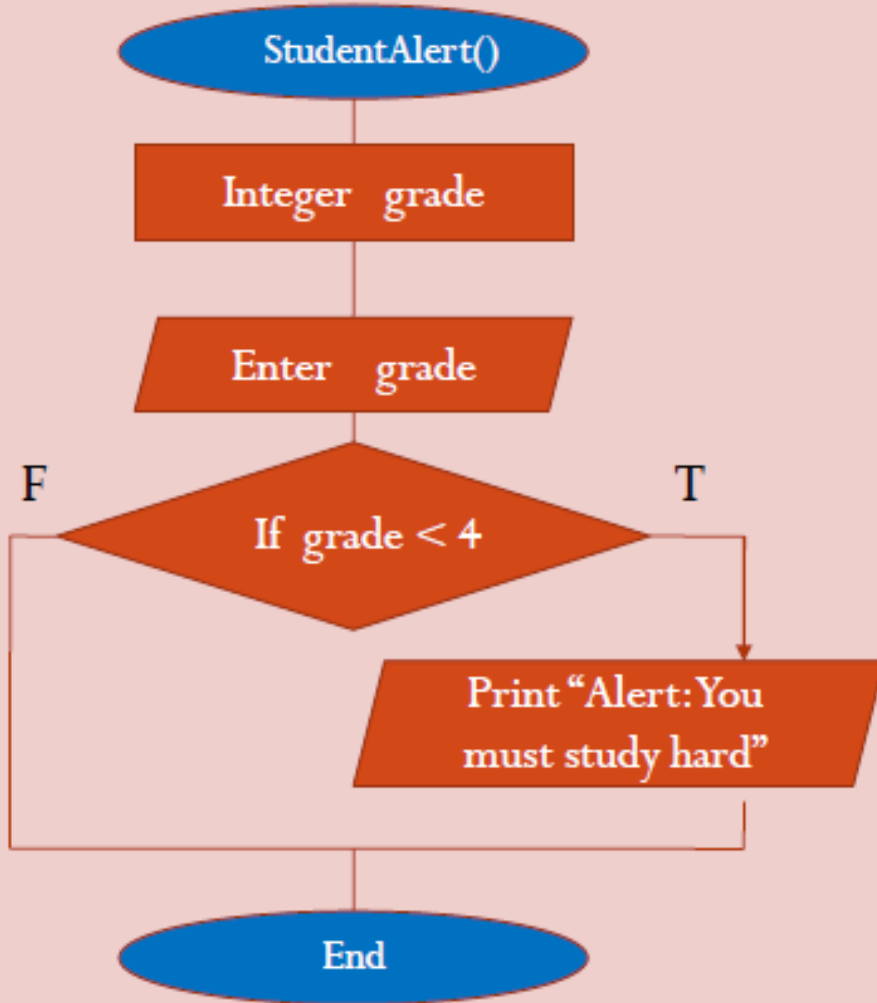


Common Forms

- **Single If/Then/Else instruction**
- **One Condition**
 - 1- The Single Selection If/Then
 - 2- The Double Selection If/Then/Else
- **Multiple Condition**
- **Multiple If/Then/Else instructions**
 - Straight-through Logic
 - Positive logic
 - Negative logic

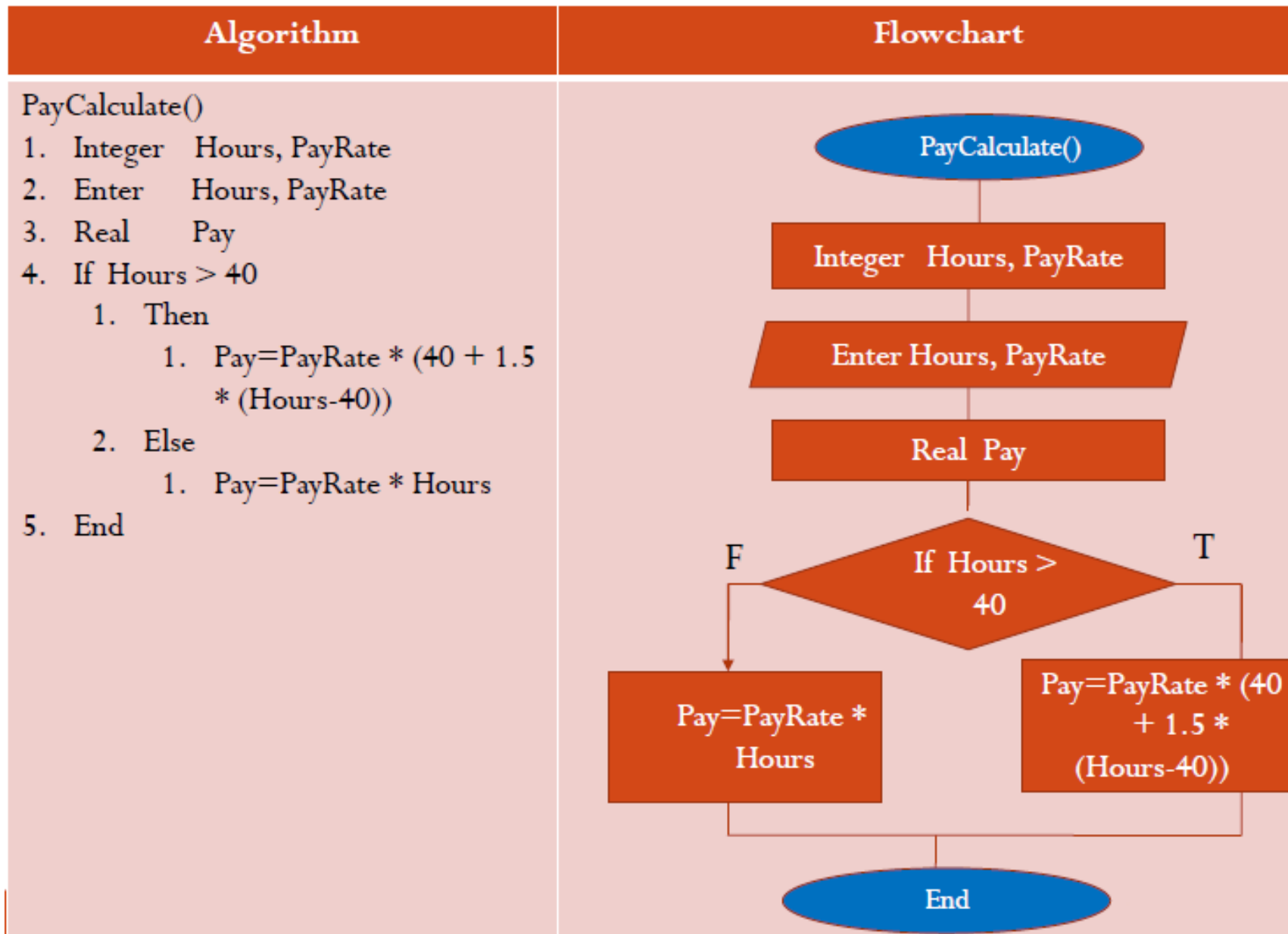
One Condition (Single Selection: If/Then)

- **One condition** and only **one instruction** (true section) no else part.
- **Example:** write an algorithm and corresponding flowchart that read a student grade on a test which is out of 10, then alerts the student if his grade is under 4.

Algorithm	Flowchart
<pre> StudentAlert() 1. Integer grade 2. Enter grade 3. If grade < 4 then 1. Print "Alert: You must study hard" 1. end </pre>	 <pre> graph TD Start([StudentAlert()]) --> Decl[Integer grade] Decl --> Input[/Enter grade/] Input --> Cond{If grade < 4} Cond -- T --> Print[/Print "Alert: You must study hard"/] Print --> End([End]) Cond -- F --> End </pre>

One Condition (Double Selection: If/Then /Else)

- **One condition** and **two instruction** (**true section** + **false section**).
- **Example:** write an algorithm and corresponding flowchart to calculate the pay at an hourly rate, and overtime pay (over 40 hours) at 1.5 times the hourly rate.
 - Hourly rate: $\text{Pay} = \text{PayRate} * \text{Hours}$
 - Overtime pay: $\text{Pay} = \text{PayRate} * (40 + 1.5 * (\text{Hours} - 40))$



Multiple Condition

- Decision in which you have **multiple conditions** that lead to **one action or set of actions** for True and False are slightly **more complicated** than those with single condition.
- In these decisions you will use **logical operators** to connect the conditions.
- The decision structure becomes **more complicated** as **the number of conditions** and/or the number of actions for a True or False resultant **increases**.

Multiple If/Then/Else Instructions

- There are three types of decision logic you will use to write algorithms for solutions consisting of more than one decision :
 1. Straight-through Logic
 2. Positive logic
 3. Negative logic

Straight-Through Logic

- Straight-through logic means that **all of the decisions are processed sequentially**, one after the other.
- There is **no Else part** of the instructions; **the False branch always goes to the next decision**, and the True branch goes to the next decision after the instructions for the True branch have been processed.
- With decisions following Straight-through logic, **all conditions are tested**. To test a condition means to process a condition to get a True or False resultant.

Straight-Through Logic (Contin.)

- Straight-through logic is **the least efficient** of all types of decision logic **Why??**
- You must use it to solve certain problems, those that require two or more **unrelated decisions**, and those in which **all decisions must be processed**, **Example:** in **data validation** (checking data to make sure it is correct)

Straight-Through Logic (Contin.)

Example: Write an algorithm to change the value of X to 0 when X becomes greater than 100, and to change the value of Y to 0 when Y becomes greater than 250.

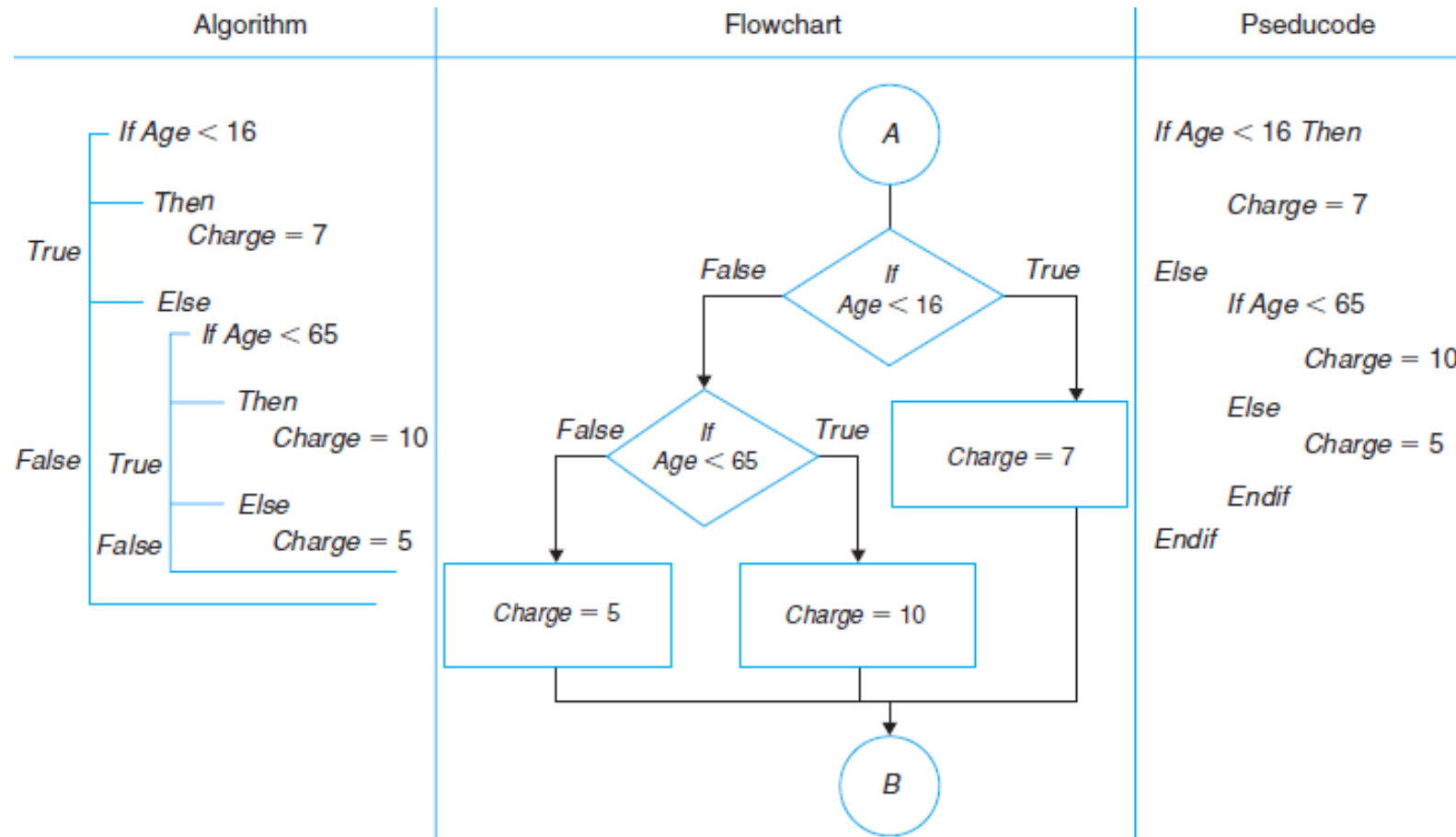
$X > 100$	$X=0$
$Y > 250$	$Y=0$

Algorithm	Flowchart
ChangeValue() 1. Integer X,Y 2. Enter X,Y 3. If $X > 100$ 1. Then 1. $X = 0$ 4. If $Y > 250$ 1. Then 1. $Y = 0$ 5. End	<pre> graph TD Start([ChangeValue()]) --> Decl[Integer X,Y] Decl --> Input[/Enter X,Y/] Input --> Cond1{If X > 100} Cond1 -- T --> SetX[X = 0] SetX --> Cond2{If Y > 250} Cond1 -- F --> Cond2 Cond2 -- T --> SetY[Y = 0] SetY --> End([End]) Cond2 -- F --> End </pre>

Positive logic

- **Positive logic** is the **easiest type** for most people to use and understand because it is **the way we think**.
- Positive logic always **uses nested If/Then/Else** instructions.
- When you use positive logic, you are telling the computer to follow a set of instructions and **continue processing if the condition is True**; if the condition is **not True**, then the **computer processes another decision**.
- When you use this logic, **no more decisions** are processed once the **resultant of a condition is True**.

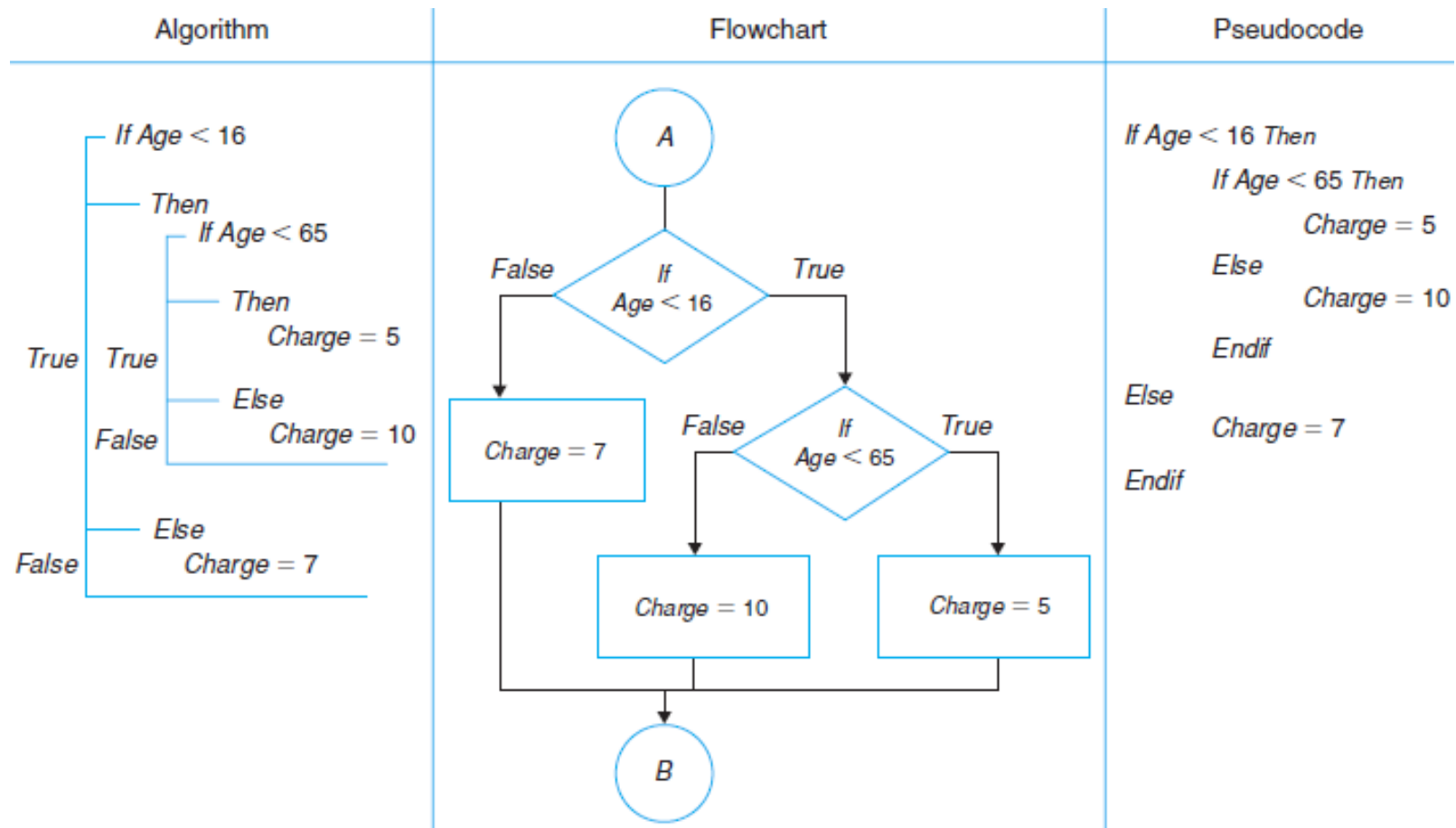
Positive logic (Contin.)



Negative logic

- **Negative logic** is similar to positive logic **except** that the flow of the processing continues through the module when the resultant of a decision is *False*.
- When you use negative logic you are telling the computer **to process another decision when the resultant of the condition is *True***; if the resultant is *False*, then the computer processes a consequent set of instructions and continues processing the module.

Negative logic (Contin.)



Logic Conversion

- Sometimes you have to change the logic from positive to negative or vice versa in order to improve the efficiency or readability of a solution.
- In a decision, there must always be instructions for a True section, but not always for a False section. If there are no instructions for the True section of a decision instruction, then it is better to convert the logic type.

Logic Conversion (Contin.)

To convert from positive logic to negative logic or vice versa, do the following:

1. Change all < to >=
2. Change all <= to >
3. Change all > to <=
4. Change all >= to <
5. Change all = to <>
6. Change all <> to =
7. Interchange all of the Then set of instructions with the corresponding Else set of instructions.

Logic Conversion (Contin.)

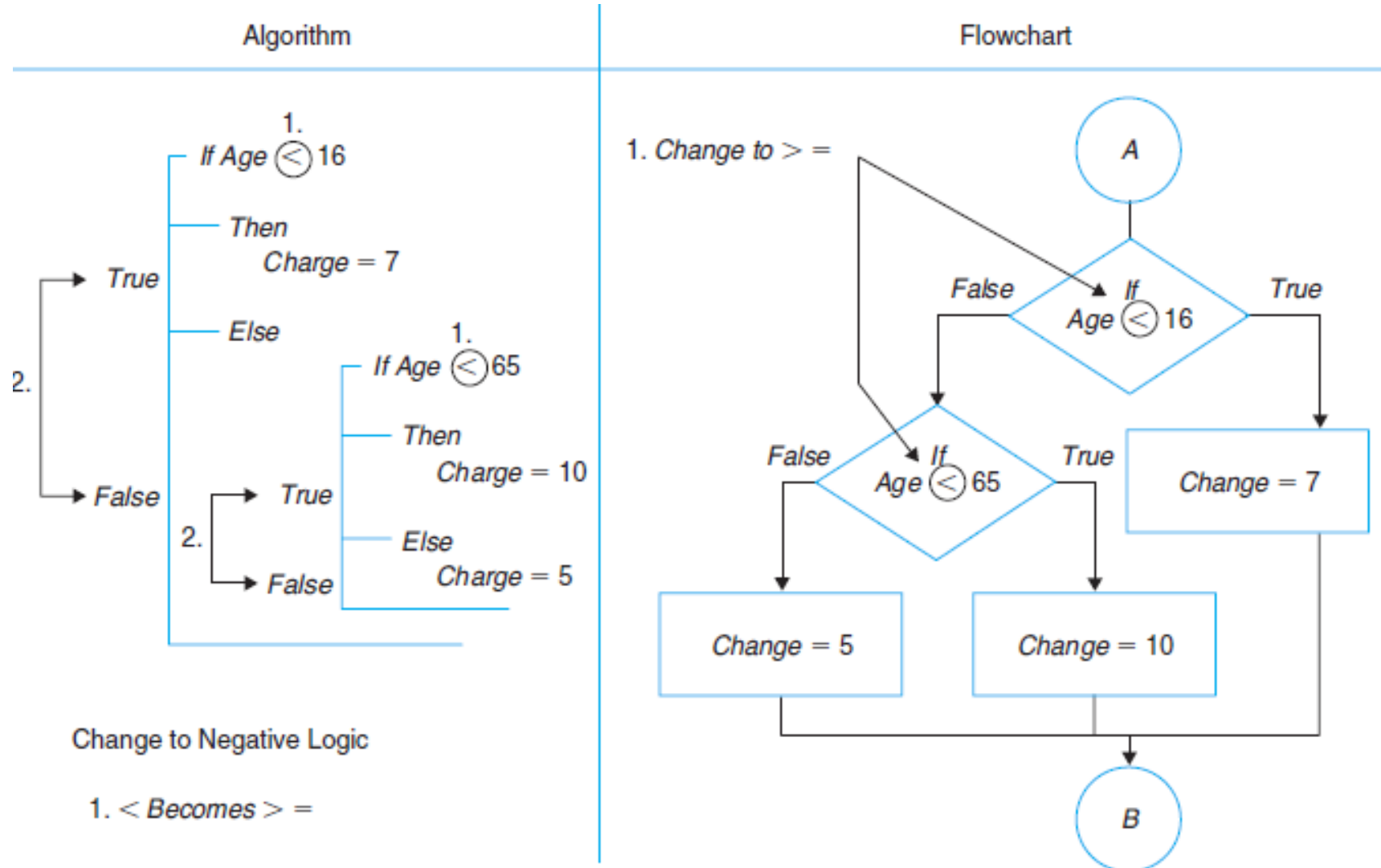
Example: Find the amount to charge people of varying ages for a concert ticket. When the person is under 16, the charge is \$7; when the person is 65 or over, the charge is \$5; all others are charged \$10.

The conditions are the following:

Age	Charge
Age < 16	\$7
Age > = 16 and Age < 65	\$10
Age > = 65	\$5

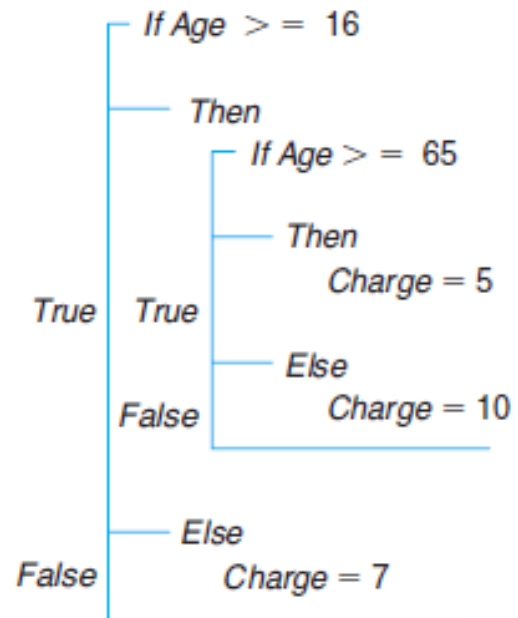
Algorithm	Flowchart
<pre> CalculateCharge() 1. Integer age , Charge 2. Enter age 3. If age < 16 1. Then 1. Charge = 7 2. Else 1. If age < 65 1. Then 1. Charge = 10 2. Else 1. Charge = 5 4. End </pre>	<pre> graph TD Start([CalculateCharge()]) --> Decl[Integer age, Charge] Decl --> Input[/Enter age/] Input --> Cond1{If age < 16} Cond1 -- T --> Charge7[Charge = 7] Cond1 -- F --> Cond2{If age < 65} Cond2 -- T --> Charge10[Charge = 10] Cond2 -- F --> Charge5[Charge = 5] Charge7 --> End([End]) Charge10 --> End Charge5 --> End </pre>

Conversion from Positive Logic to Negative Logic

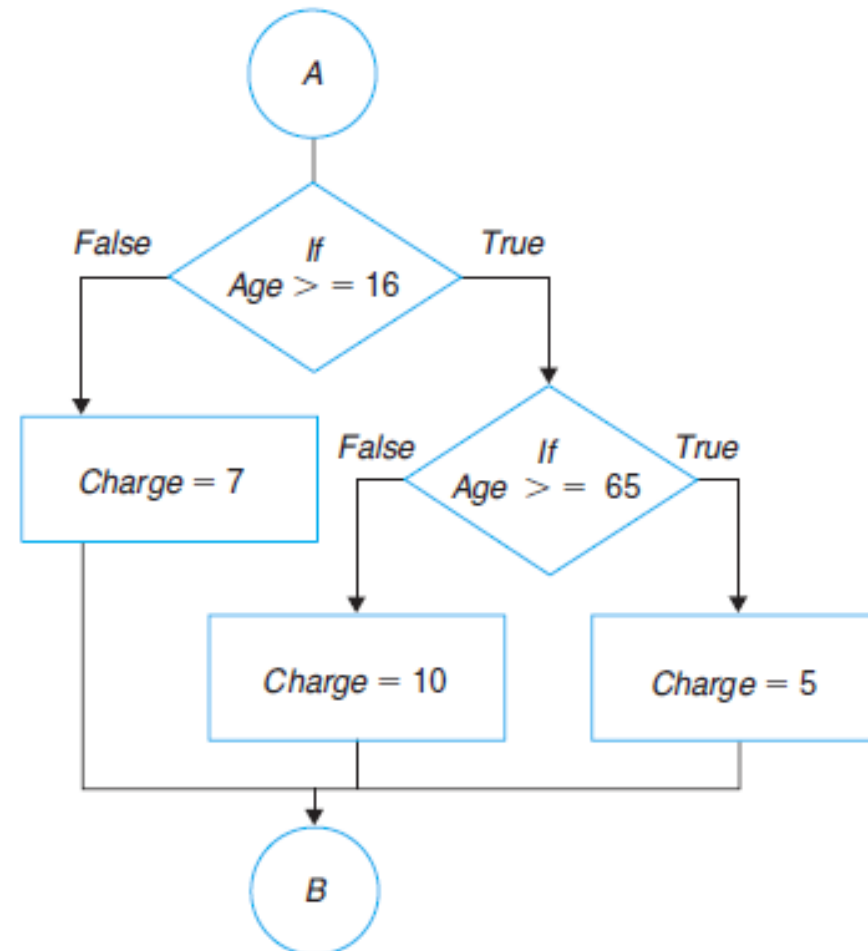


Conversion from Positive Logic to Negative Logic

2. The true instructions exchange with the false instructions



2. Exchange the true sides with the false sides



Decision Table

- A good way to simplify the process of discovering complicated actions and conditions is to draw a decision table.
- A Decision table consist of 4 parts:
 1. The conditions
 2. The actions
 3. The combinations of True and False for the conditions
 4. The action to be taken or the consequences for each combination of conditions.

Decision Table Format

List of Conditions	Condition1	True	True	True	True	False	False	False	False	All Possible Combinations of True & False
	Condition2	True	True	False	False	True	True	False	False	
	Condition3	True	False	True	False	True	False	True	False	
List of Actions	Action1		X				X			Consequences
	Action2	X			X	X			X	
	Action3			X				X		

Note: The total number of possible combinations of **True** or **False** for the conditions is $2^{\text{\#conditions}}$

Decision Table Example

Set up a decision table for a store policy for charging a purchase.

There are three conditions:

1. The purchase is less than \$100.
2. The last payment to the account was made in the last 30 days.
3. The balance of the account is less than \$1000.

The following actions could be taken:

1. Credit is okay, and the customer can charge the item.
2. Refer the customer to the credit department.
3. Credit is denied and the customer cannot charge the item.

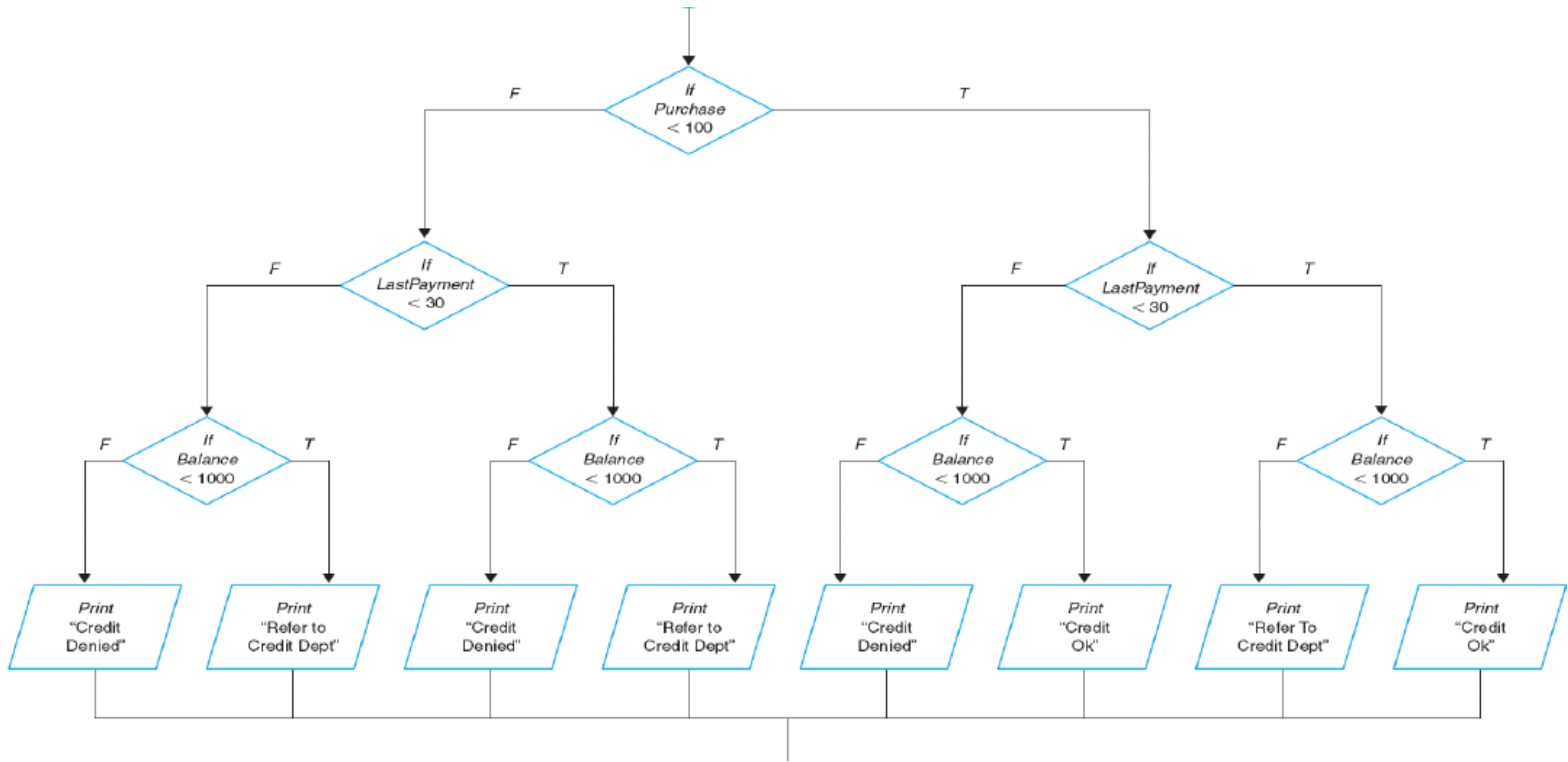
Decision Table Example (Contin.)

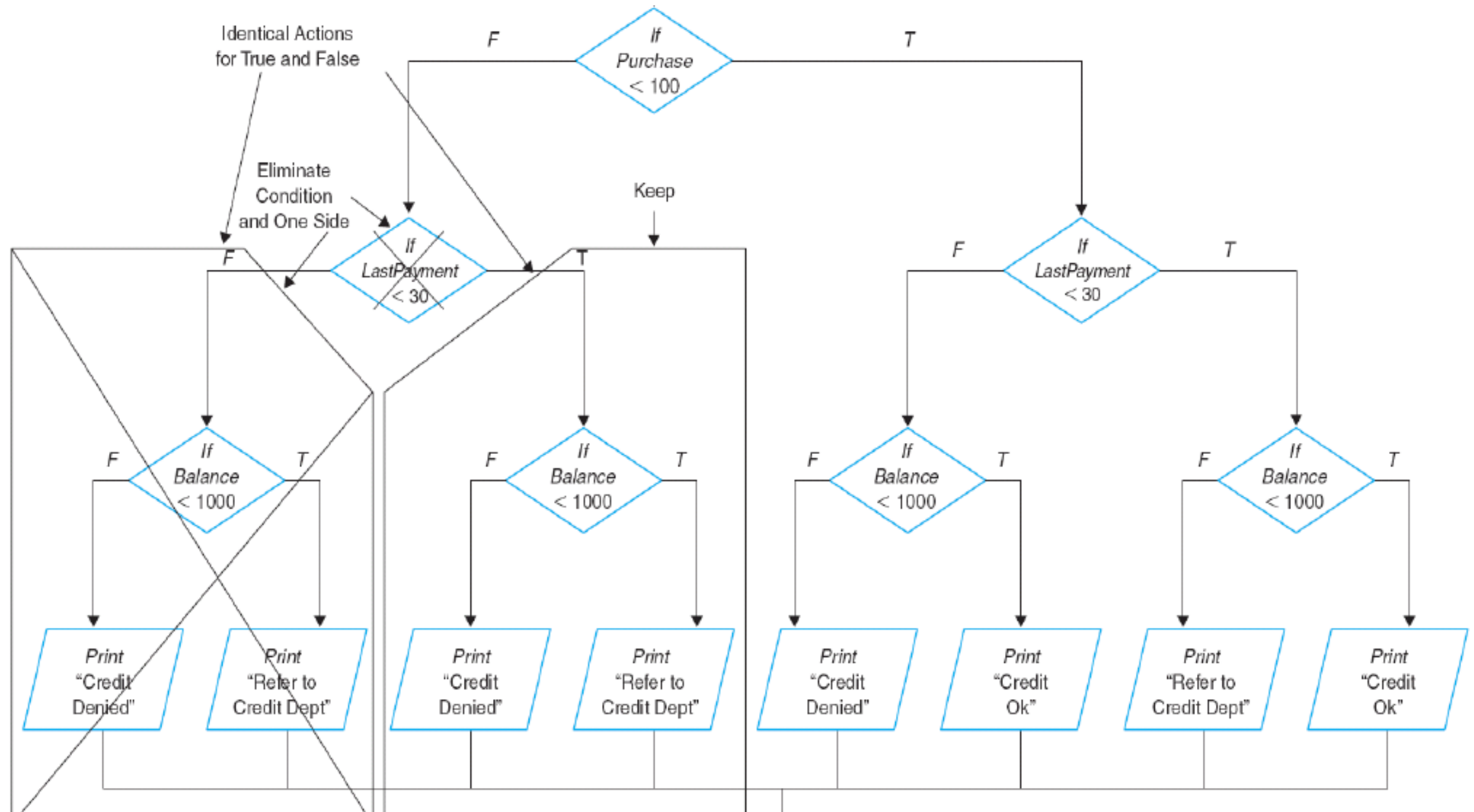
List of Conditions	1	<i>Purchase < \$100</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>False</i>	<i>False</i>	<i>False</i>	<i>False</i>
	2	<i>LastPaymentWithin30Days</i>	<i>True</i>	<i>True</i>	<i>False</i>	<i>False</i>	<i>True</i>	<i>True</i>	<i>False</i>	<i>False</i>
	3	<i>Balance < \$1000</i>	<i>True</i>	<i>False</i>	<i>True</i>	<i>False</i>	<i>True</i>	<i>False</i>	<i>True</i>	<i>False</i>
List of Actions	1	<i>CreditOk</i>	<i>X</i>		<i>X</i>					
	2	<i>ReferToCreditDepartment</i>		<i>X</i>			<i>X</i>		<i>X</i>	
	3	<i>CreditDenied</i>				<i>X</i>		<i>X</i>		<i>X</i>

Flowchart development from the decision table

The four steps to develop a flowchart from the decision table are:

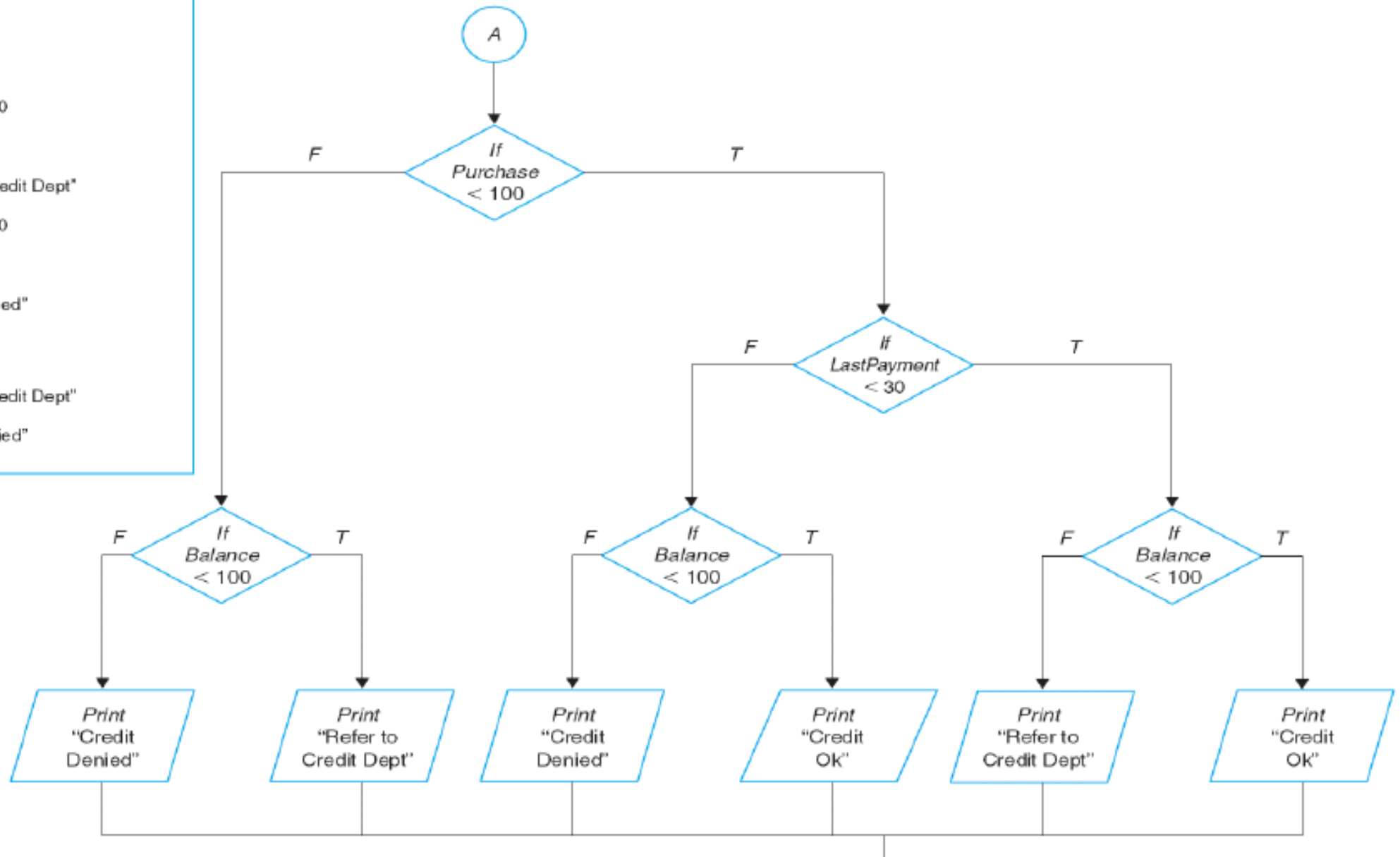
1. Draw all decisions in flowchart form.
2. Compare the true and false sides of each decisions, starting with the first one.
3. Eliminate any decisions that have the same instructions on both the true and false sides, keeping the true consequence or action.
4. Redraw the flowchart.





Algorithm:

```
T If Purchase < 100
  Then
    If LastPayment < 30
      Then
        If Balance < 1000
          Then
            Print "Credit Ok"
          Else
            Print "Refer to Credit Dept"
        Else
          If Balance < 1000
            Then
              Print "Credit Ok"
            Else
              Print "Credit Denied"
      Else
        If Balance < 1000
          Then
            Print "Refer to Credit Dept"
          Else
            Print "Credit Denied"
```



Questions?
